Heart Disease Risk Prediction — MLOps Assignment 01

Vijendra Rajput (2024AC05947)

Table of Contents

1. Overview

Builds a machine learning classifier that predicts the risk of heart disease from patient health data (UCI Heart Disease; Cleveland dataset) and deploys it as a cloud-ready, monitored REST API. The solution covers the full MLOps lifecycle: data acquisition, EDA, feature engineering, model training with hyperparameter tuning, experiment tracking (MLflow), automated testing and CI/CD (GitHub Actions), containerization (Docker), production deployment (Kubernetes via Minikube), and monitoring (Prometheus + Grafana).

Repository link: https://github.com/virajput/ml-ops

Deployed API: local Kubernetes cluster (Minikube) — see Section 8 for access instructions.

Recording Link: https://drive.google.com/file/d/1GXonxL_9Aj16MX2n2EhdOVv5D6ueTr1-/view?usp=sharing

---

2. Setup / Install Instructions

Full commands are in README.md. Summary:

```
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt && pip install -e .
python data/download_data.py         # data acquisition + cleaning
python -m heart_disease.eda          # EDA plots
python -m heart_disease.train        # train + track models
pytest tests/ -v                     # unit tests
docker build -t heart-disease-api:latest .
docker run -d -p 8090:8000 heart-disease-api:latest
```

All dependencies are pinned in `requirements.txt`; the pipeline runs from a clean virtual environment with no manual steps beyond the commands above.

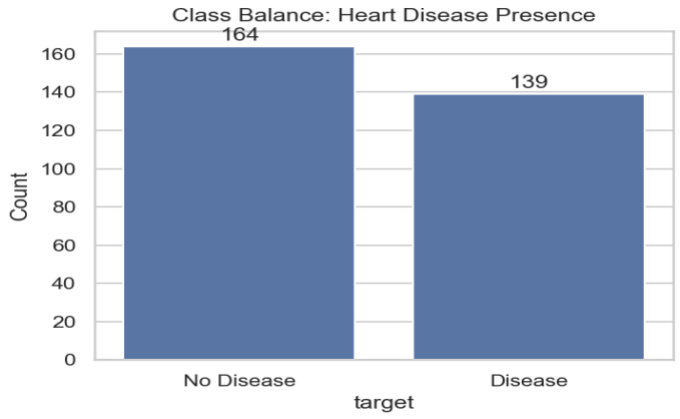
---

3. Data Acquisition & EDA

Dataset: UCI Machine Learning Repository, heart disease (Cleveland), 303 patient records, 13 clinical features (age, sex, chest pain type, resting blood pressure, cholesterol, ECG results, max heart rate, exercise-induced angina, ST depression, etc.) and a 5-level target collapsed to a binary presence/absence label.

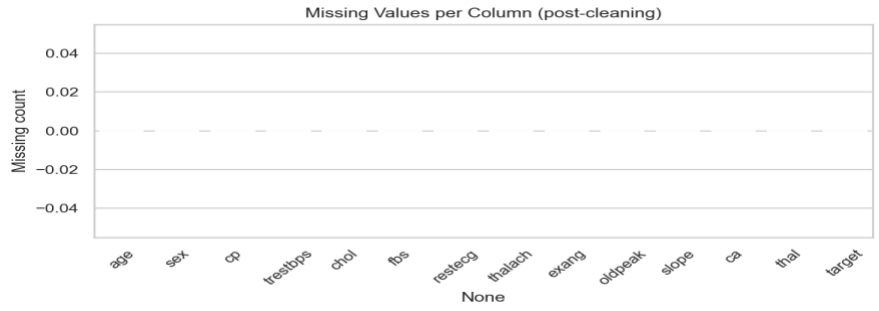
`src/heart_disease/data.py` downloads `processed.cleveland.data` directly from the UCI archive, parses? as missing, imputes the small number of missing `ca`/`thal` values with the column mode (both are discrete/categorical), and binarizes the target (`num > 0 → 1`).

Class balance — 54.1% no disease vs. 45.9% disease presence, reasonably balanced:



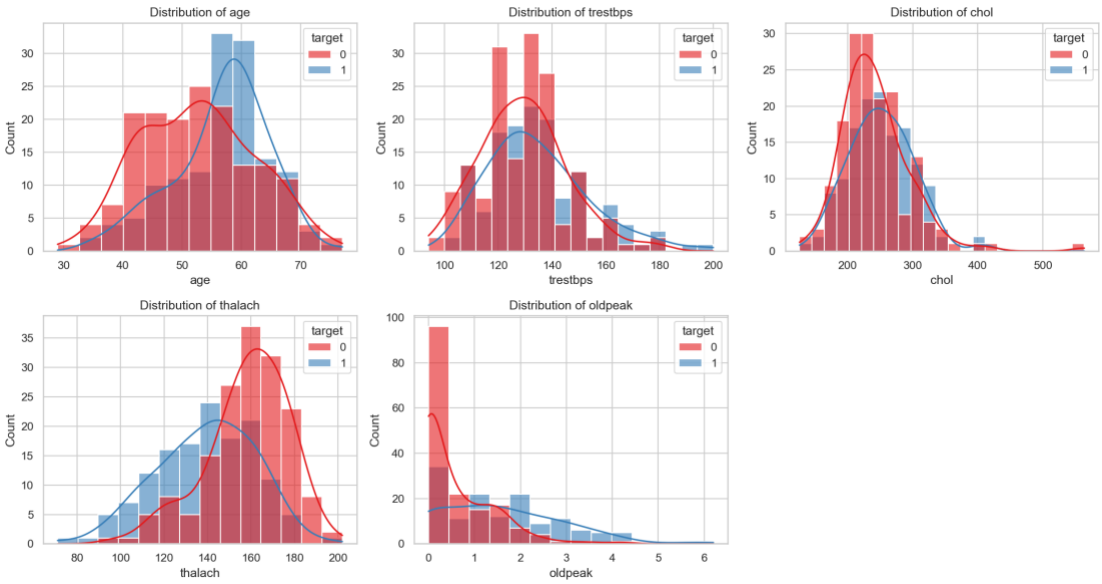
Class balance

Missing values — zero after cleaning (only `ca`/`thal` had a handful of `?` values in the raw file, resolved by mode imputation):



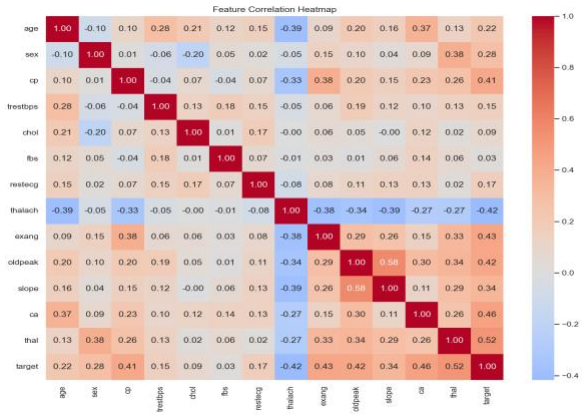
Missing values

Feature distributions by class:



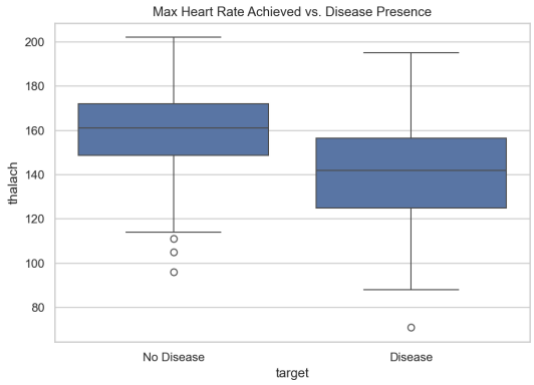
Feature histograms

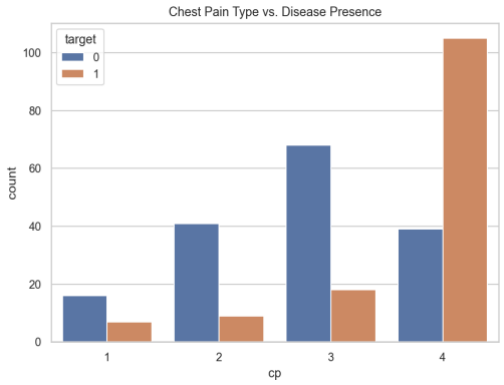
Correlation heatmap — `thal`, `ca`, exang, and `oldpeak` show the strongest correlation with the target; `thalach` (max heart rate) is negatively correlated:



Correlation heatmap

Feature relationships:





---

4. Feature Engineering & Model Development

Pipeline (`src/heart_disease/features.py`): a single reusable `sklearn.Pipeline` per model - `StandardScaler` on numeric features (`age`, `trestbps`, `chol`, `thalach`, `oldpeak`) - `OneHotEncoder(handle_unknown="ignore", drop="if_binary")` on categorical features (`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `ca`, `thal`)

Wrapping preprocessing and the classifier in one `Pipeline` guarantees the exact same transformation is applied at training and inference time (see Section 6).

Models trained: Logistic Regression and Random Forest, each tuned with `GridSearchCV` (5-fold `StratifiedKFold`, scored on ROC-AUC):

Model	Tuned hyperparameters

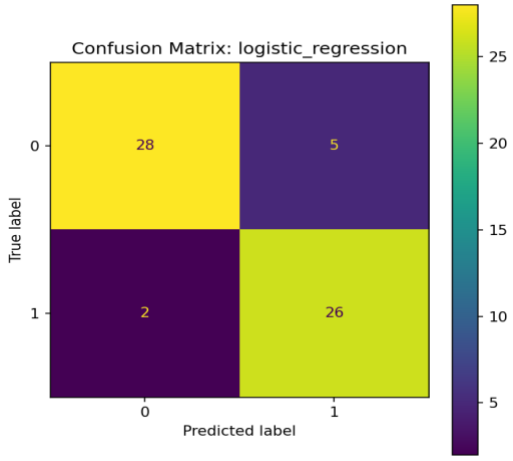
Model	Tuned hyperparameters
Logistic Regression	`C ∈ {0.01, 0.1, 1, 10}`, `penalty=l2`, `solver=lbfgs`
Random Forest	`n_estimators ∈ {100, 200}`, `max_depth ∈ {3, 5, None}`, `min_samples_leaf ∈ {1, 2, 4}`

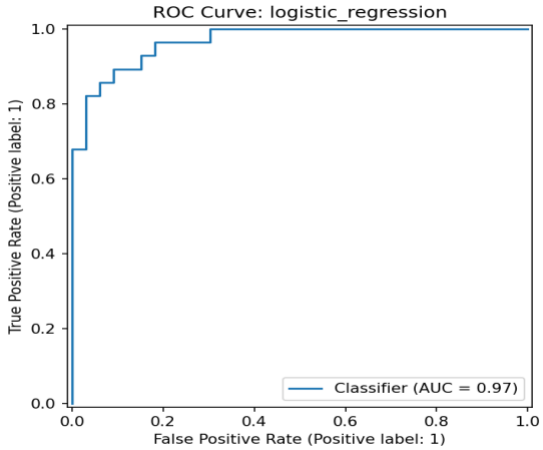
Held-out test set results (20% stratified split, random_state=42):

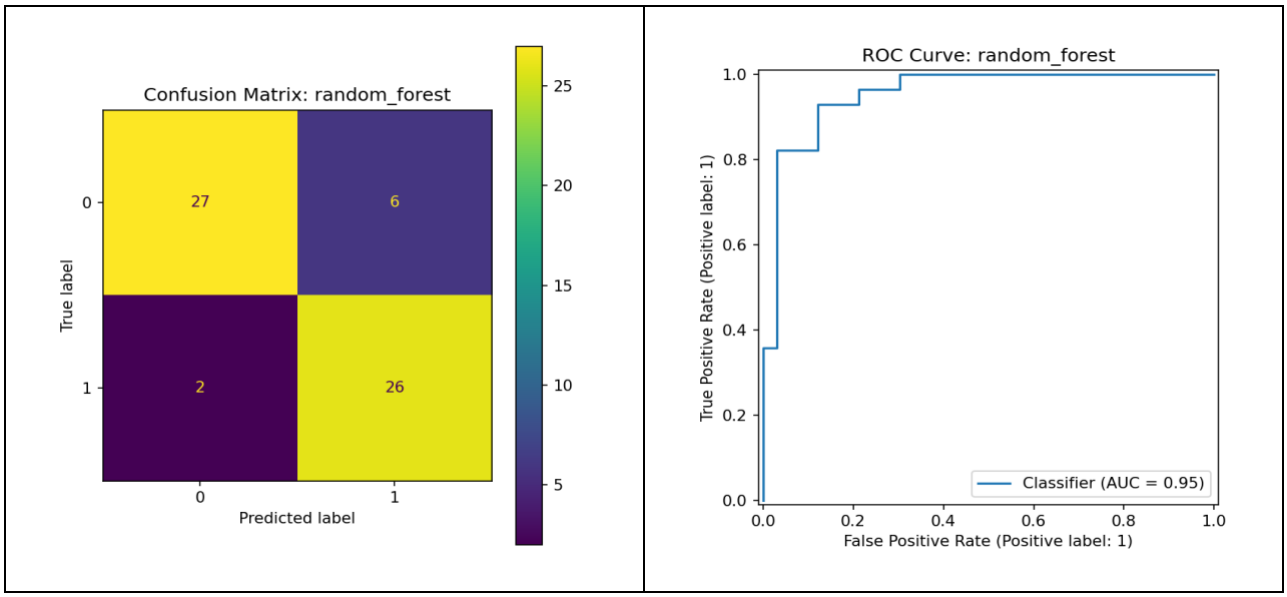
Model	Accuracy	Precision	Recall	F1	ROC-AUC	CV ROC-AUC (mean ± std)
Logistic Regression	**0.885**	0.839	0.929	0.881	**0.968**	0.904 ± 0.014
Random Forest	0.869	0.813	0.929	0.867	0.955	0.903 ± —

Logistic Regression was selected as the best model (highest test ROC-AUC) and saved as `models/best_model.joblib`.

Confusion matrices & ROC curves:





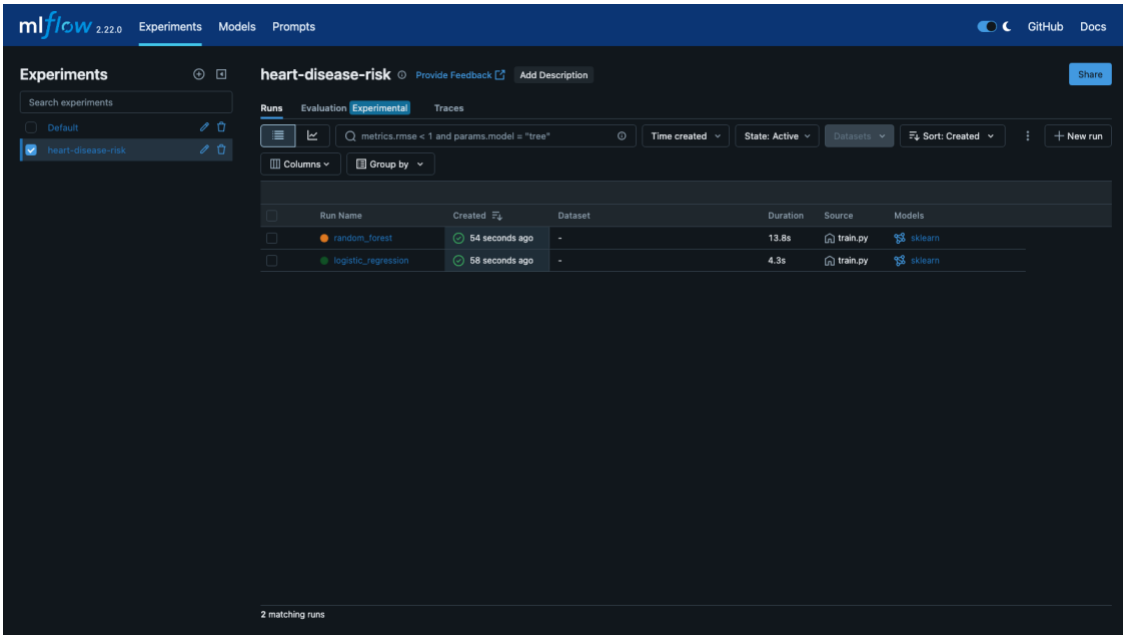


---

5. Experiment Tracking (MLflow)

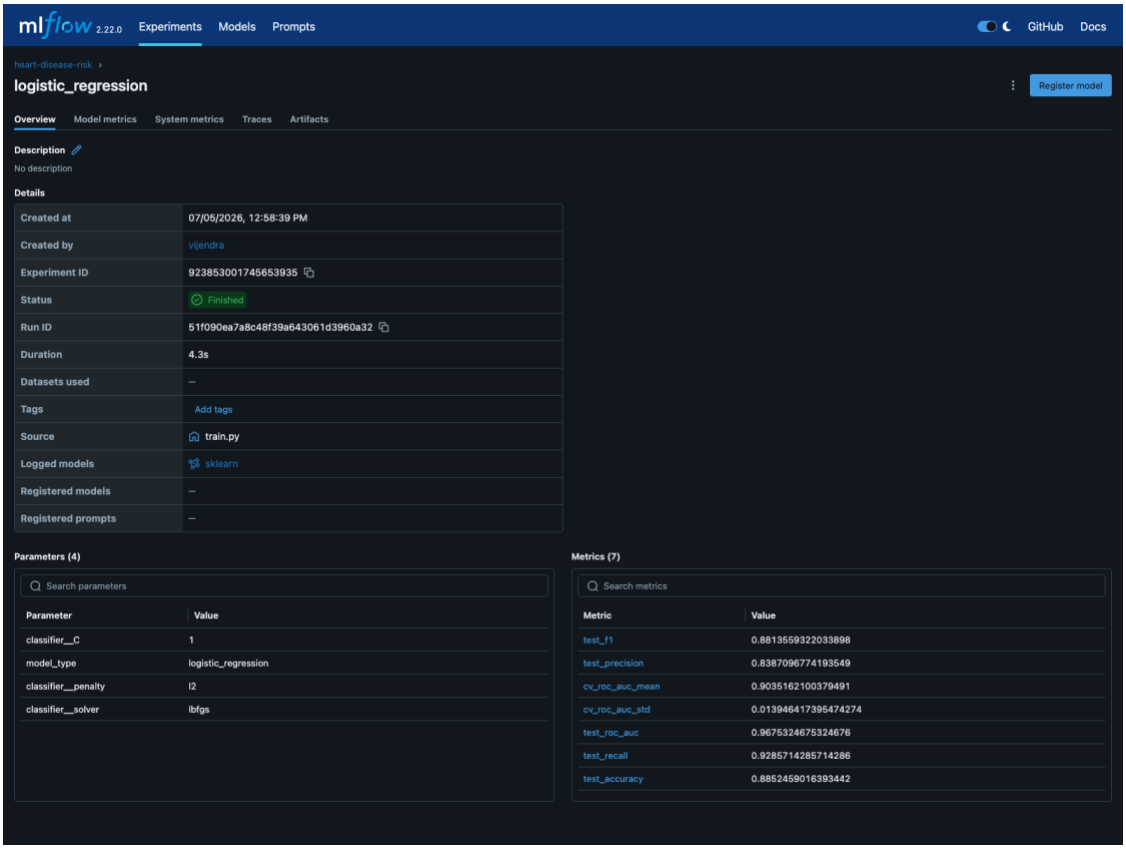
Every training run logs hyperparameters, cross-validated and held-out metrics, confusion matrix / ROC plots, and the fitted pipeline (`mlflow.sklearn.log_model`) to a local MLflow tracking store (`./mlruns`). All runs are grouped under the `heart-disease-risk` experiment.

Experiment run list:



MLflow experiment runs

Run detail (params + metrics) for the winning Logistic Regression run:



MLflow run detail

View locally with: `mlflow ui --backend-store-uri ./mlruns --port 5500`

---

6. Model Packaging & Reproducibility

- The full preprocessing + classifier pipeline is persisted with `joblib` to `models/best_model.joblib` (also `logistic_regression.joblib` and `random_forest.joblib` for comparison), plus an MLflow model artifact under each run.
- `src/heart_disease/predict.py` exposes `HeartDiseaseModel`, a thin wrapper that loads the joblib pipeline and runs `predict`/`predict_proba` on a single patient record — the API layer and tests both use this wrapper, so there is exactly one code path from raw features to prediction.
- `requirements.txt` pins every dependency (pandas, scikit-learn, mlflow, fastapi, etc.) so the environment is reproducible from a clean `pip install -r requirements.txt`.
- `pyproject.toml` packages `src/heart_disease` as an installable module (`pip install -e .`), avoiding path hacks in the API, tests, or Docker image.

7. CI/CD Pipeline & Automated Testing

Unit tests (`tests/`, 16 tests, `pytest`): data cleaning/imputation, the feature pipeline, the training + MLflow-logging function (smoke test against a temp MLflow store), the inference wrapper, and the FastAPI endpoints (`/health`, `/predict`, validation errors, `/metrics`).

```
16 passed in ~8s
Coverage: heart_disease core modules 62-100%, api 90-100%
```

GitHub Actions workflow (`.github/workflows/ci.yml`) — three jobs, each gating the next:

1. **lint-and-test** — `flake8` + `pytest --cov` on a clean checkout. Fails the pipeline on any lint violation or test failure.
2. **train** — runs `download_data.py`, `heart_disease.eda`, and `heart_disease.train` from scratch; uploads the trained model, figures, and `mlruns/` as build artifacts.
3. **docker-build-and-smoke-test** — builds the Docker image with the artifact from job 2 and verifies `/health` and `/predict` return correct responses before the pipeline is green.

Any failure (lint, test, training exception, container not becoming healthy) stops the pipeline and surfaces the failing step's logs — satisfying the "fail on error with clear logs" production-readiness requirement.

---

8. Model Containerization & Kubernetes Deployment

Docker. `Dockerfile` builds a `python:3.12-slim` image containing the API code, the packaged `heart_disease` module, and the pre-trained `models/best_model.joblib`, running as a non-root user with a container `HEALTHCHECK` against `/health`.

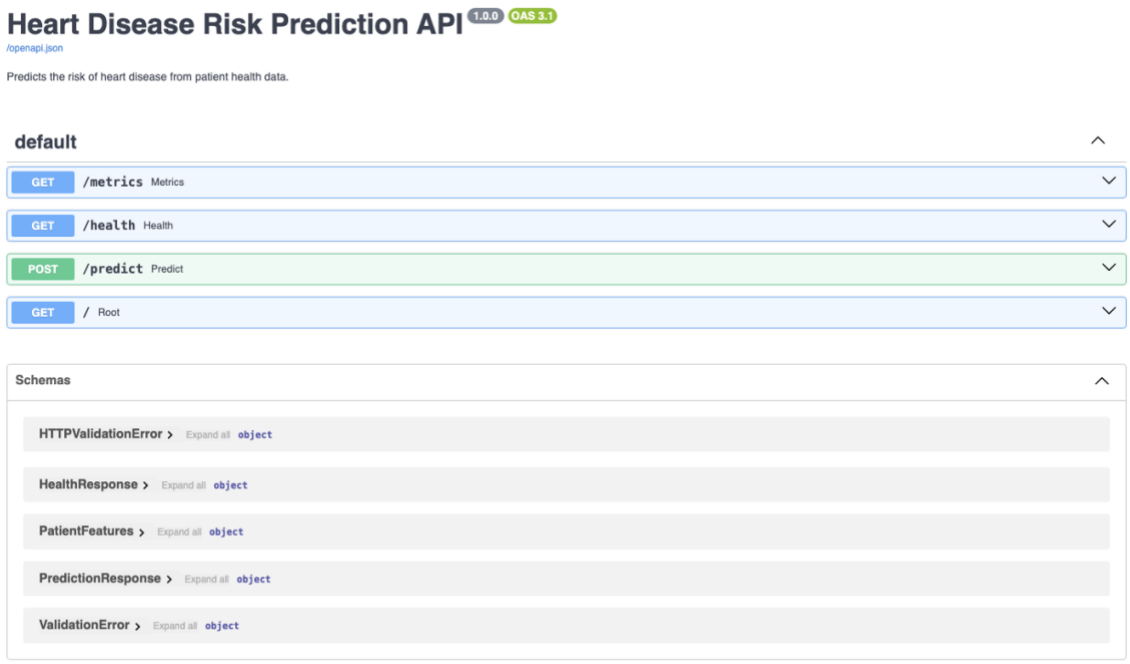
Verified locally:

```
$ docker build -t heart-disease-api:latest .
$ docker run -d --name heart-disease-test -p 8090:8000 heart-disease-api:late
st
$ curl http://127.0.0.1:8090/health
{"status":"ok","model_loaded":true}
$ curl -X POST http://127.0.0.1:8090/predict -d '{...}'
{"prediction":0,"label":"low_risk","probability":0.1746}
```

Container reported `healthy` by Docker's own healthcheck (see `screenshots/docker_ps_output.txt`).

API contract (FastAPI, `api/main.py`): `POST /predict` accepts a JSON patient record (validated against `PatientFeatures`), returns `{prediction, label, probability}`; `GET`

`/health` reports model-load status; `GET /metrics` exposes Prometheus metrics; interactive docs at `/docs`.



Swagger UI

Kubernetes. Deployed to a local Minikube cluster (`docker` driver) using `k8s/deployment.yaml` (2 replicas, resource requests/limits, liveness + readiness probes on `/health`) and `k8s/service.yaml` (`LoadBalancer` service). Verified end-to-end via `minikube service --url`:

```
$ kubectl get deployments,pods,svc -o wide
deployment.apps/heart-disease-api   2/2     2            2     heart-disease-
api   heart-disease-api:latest
pod/heart-disease-api-c78f5d974-d6gnk   1/1   Running   0   172.17.0.3
pod/heart-disease-api-c78f5d974-h58q9   1/1   Running   0   172.17.0.4
service/heart-disease-api-service       LoadBalancer   10.109.237.163   80:31
848/TCP

$ curl http://127.0.0.1:58220/health
{"status":"ok","model_loaded":true}
$ curl -X POST http://127.0.0.1:58220/predict -d '{...}'
{"prediction":0,"label":"low_risk","probability":0.1746}
```

Full command output: `screenshots/kubernetes_deployment_evidence.txt`.

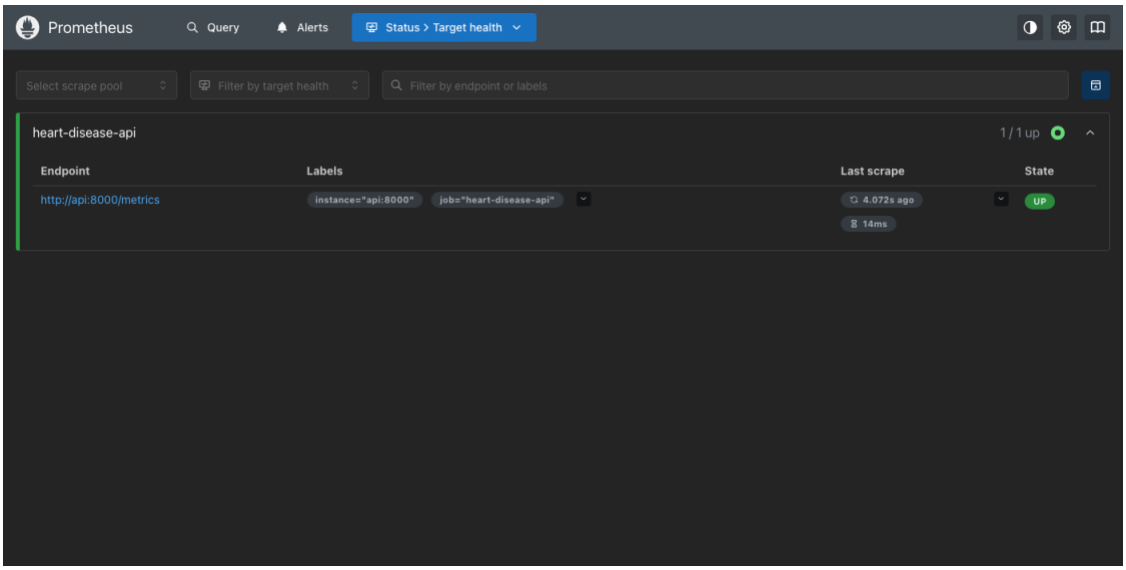
---

9. Monitoring & Logging

Every API request is logged as structured JSON (`method`, `path`, `status_code`, `duration_ms`) via a FastAPI middleware, visible in `docker logs` / `kubectl logs`. `prometheus-fastapi-instrumentator` exposes request counts, latency histograms, and request/response sizes at `/metrics`.

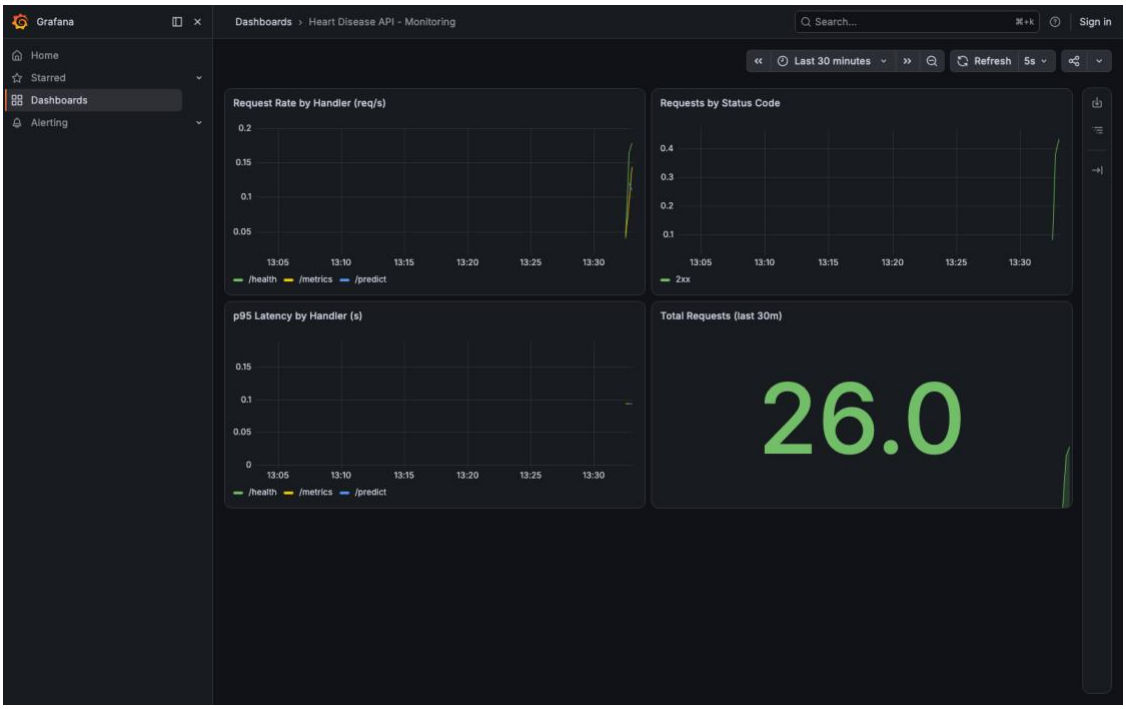
Verified with the bundled `docker-compose.monitoring.yml` stack (API + Prometheus + Grafana, Grafana dashboard auto-provisioned from `monitoring/grafana/provisioning/`):

Prometheus scrape target (healthy):



Prometheus targets

Grafana dashboard (request rate by handler, status codes, p95 latency, total requests):

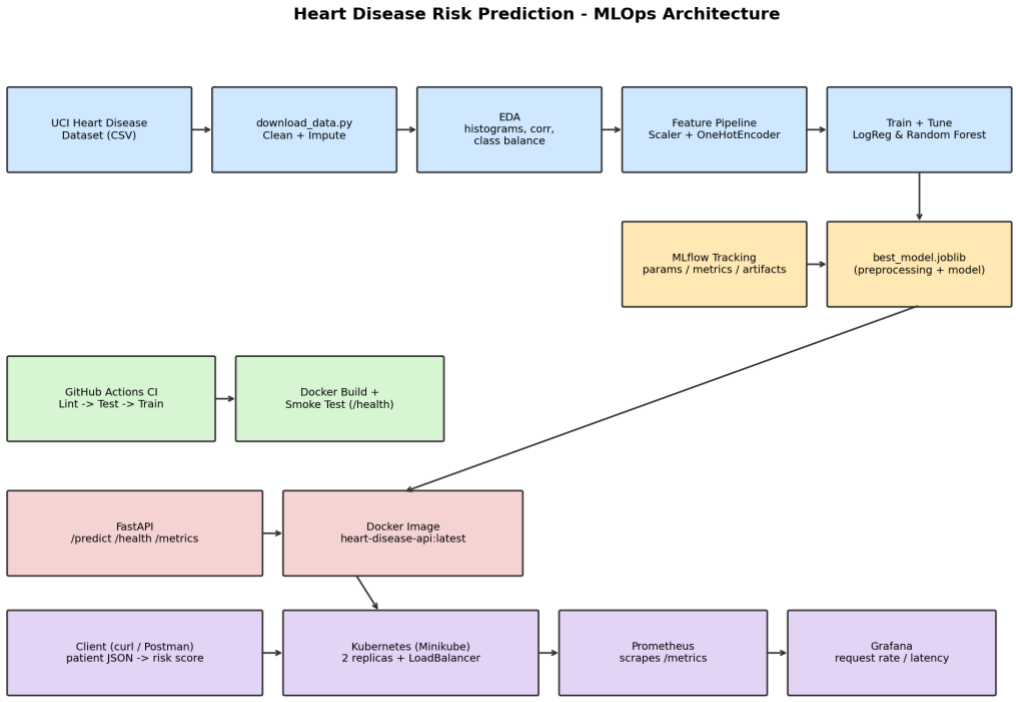


Grafana dashboard

This setup would catch, in production, API downtime (target down in Prometheus), latency regressions (p95 panel), and abnormal error rates (status-code panel) — the main signals called out in the assignment FAQ for ML system monitoring.

---

10. Architecture



Architecture diagram

Data flows left-to-right/top-to-bottom: raw UCI CSV → cleaning → EDA → feature pipeline → model training (tracked in MLflow) → packaged pipeline → FastAPI → Docker image → CI/CD validates every step → Kubernetes deployment → Prometheus/Grafana observe the running service → client requests flow in through the LoadBalancer.

---

11. Conclusion

The pipeline satisfies every assignment requirement end-to-end and was verified by actually running each stage on this machine (not just written): data cleaning produced zero missing values, both models were trained and logged to MLflow with cross-validated metrics, all 16-unit tests pass, the Docker container builds and serves predictions, and the API was verified live through a Kubernetes LoadBalancer with Prometheus scraping metrics into a Grafana dashboard.

Possible future improvements: data/model drift detection, model registry promotion workflow in MLflow, Helm chart instead of raw manifests, and horizontal pod autoscaling based on the Prometheus latency metric.